

# Basiswissen Softwaretest (Linz/Spillner)

Zusammenfassung (in Arbeit)

Patrick Bucher

16.08.2025

## Inhaltsverzeichnis

|          |                                                                        |          |
|----------|------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Einleitung</b>                                                      | <b>1</b> |
| 1.1      | Kapitelübersicht . . . . .                                             | 2        |
| <b>2</b> | <b>Grundlagen des Softwaretestens</b>                                  | <b>2</b> |
| 2.1      | Begriffe und Motivation . . . . .                                      | 2        |
| 2.1.1    | Fehlerbegriff . . . . .                                                | 3        |
| 2.1.2    | Testbegriff . . . . .                                                  | 5        |
| 2.1.3    | Grundsätze des Testens . . . . .                                       | 6        |
| 2.2      | Softwarequalität: Qualitätsmanagement und Qualitätssicherung . . . . . | 6        |

## 1 Einleitung

Software ist heutzutage allgegenwärtig und sie trägt nicht nur zum Funktionieren unserer Welt bei, von ihr hängt auch immer mehr unsere Sicherheit ab. Nicht nur die Abwicklung von Geschäftsprozessen hängt von Software ab, sondern ihre Erweiterbarkeit gibt auch vor, wie schnell eine Firma ihre Geschäftstätigkeit ausbauen kann. Die Qualität von Software ist ein entscheidender Faktor für den Erfolg von Produkten und Firmen.

Systematisches Testen von Software hilft Unternehmen dabei, die Qualität ihrer Softwaresysteme zu erhöhen. Das vorliegende Buch stellt das hierzu notwendige Grundlagenwissen bereit. Es richtet sich an Tester und Entwickler – sowie an alle, die im Rahmen der agilen Softwareentwicklung Testaufgaben übernehmen. Es richtet sich an Lehrende und Lernende gleichermaßen.

Das *International Software Testing Qualifications Board* (ISTQB) koordiniert die Zertifizierung im Bereich Software-Qualitätssicherung im Rahmen des ISTQB-Schemas und wird durch länderspezifische Gremien (wie z.B. durch das *Swiss Testing Board*) ergänzt. Die drei Ausbildungs-

stufen *Foundation*, *Advanced* und *Expert* werden dabei um zusätzliche Spezialistenmodule (u.a. fürs Testen im agilen Kontext) ergänzt.

## 1.1 Kapitelübersicht

Das vorliegende Buch deckt den Stoff bis zur ersten Stufe (*Foundation*) ab und behandelt in den folgenden Kapiteln diese Themen:

- **Kapitel 2** erörtert die Grundlagen des Softwaretests. Neben dem *Warum*, dem *Wann*, dem *Wozu* und dem *Wie* wird auf das Konzept des Testprozesses und auf die notwendigen Kompetenzen beim Testen eingegangen.
- **Kapitel 3** erläutert die Rolle des Testens in verschiedenen Entwicklungsmodellen (sequentiell, agil), die verschiedenen Teststufen und -arten, die Unterschiede zwischen funktionalen und nicht-funktionalen Tests, Regressionstests und Ansätze zur Verbesserung der Testautomatisierung.
- **Kapitel 4** behandelt statische Testverfahren, bei denen das Testobjekt nicht ausgeführt wird.
- **Kapitel 5** erörtert dynamische Tests und deren Einordnung in *Blackbox*- und *Whitebox*-Verfahren mit den dazugehörigen Testverfahren und -methoden.
- **Kapitel 6** behandelt die Organisation des Testprozesses und die dazu notwendigen Qualifikationen der involvierten Mitarbeitern. Nebst den Elementen einer Teststrategie werden auch Verfahren zur Aufwands- und Kostenschätzung des Softwaretests erläutert. Risikobasiertes Testen, Fehler- und Konfigurationsmanagement und Wirtschaftlichkeit sind ebenfalls Themen dieses Kapitels.
- **Kapitel 7** stellt verschiedene Arten von Testwerkzeugen vor und gibt Hinweise zu deren Auswahl und Einführung.

## 2 Grundlagen des Softwaretestens

In diesem Kapitel werden die Grundbegriffe des Softwaretestens eingeführt, etablierte Grundsätze des Testens vorgestellt und die Aktivitäten des Testprozesses erläutert.

### 2.1 Begriffe und Motivation

Industriell hergestellte Produkte werden zumeist durch Stichproben geprüft, was bei Softwareprodukten, die immateriell sind, nicht gleich funktioniert. Fehler in Software kosten nicht nur Zeit und Geld, sondern können auch den Ruf einer Organisation schädigen oder im Extremfall sogar zum Tod von Menschen führen.

Durch das Testen von Software kann deren Qualität eingeschätzt werden, und das Risiko unentdeckter Fehler, die sonst erst im Produktiveinsatz der Software zutage treten würden, kann minimiert werden. Beim Testen von Software sollen alle Beteiligten des Projekts involviert sein.

Beim – statischen und dynamischen – Testen von Softwarekomponenten werden deren Fehler (genauer: Fehlerzustände bzw. Fehlerwirkungen) erkannt.

Beim *dynamischen* Testen kommt das *Testobjekt* (d.h. die Software) zur stichprobenartigen Ausführung, wozu das Testobjekt mit *Testdaten* versehen und einzelne Testfälle darauf ausgeführt werden, wonach geprüft wird, ob das beobachtete Ergebnis den Anforderungen entspricht.

Der gesamte Testprozess umfasst jedoch noch viele weitere Aktivitäten wie z.B. das Planen des Testvorgangs; das Abschätzen des Testaufwands; Analyse, Design und Umsetzung der Tests; das Erstellen von Berichten über Testfortschritt, Testergebnisse, Qualitätsbeurteilung und Risikobewertung.

Die Aktivitäten und Dokumentationen werden i.d.R. zwischen Auftraggeber und -nehmer ausgehandelt und unterliegen teilweise gesetzlichen Vorgaben oder Standards. Die Testaktivitäten unterscheiden sich im Lebenszyklus der Software; oftmals markieren Tests den Übergang von einer Phase in die nächste (z.B. die Freigabe einer neuen Version).

Obwohl Testaktivitäten von Werkzeugen abhängen, ist das Testen v.a. eine intellektuelle Tätigkeit, die Fachwissen und verschiedene Fähigkeiten erfordert. Neben der ausführbaren Software können im Rahmen von statischen Tests auch andere Artefakte wie z.B. Dokumentation, Anforderungen und Quellcode Testobjekte sein.

Je früher Fehler gefunden werden (z.B. bereits in den Anforderungen), desto besser ist das für den weiteren Entwicklungsprozess. Beim Testen wird auch geprüft, ob sich das System gemäss den Wünschen und Vorstellungen der Benutzer verhält. Es ist sinnvoll aber nicht immer machbar, die Benutzer im Rahmen einer Validierung möglichst im gesamten Entwicklungszyklus zu involvieren.

Ab einer gewissen Komplexität gibt es praktisch keine Softwaresysteme, die völlig fehlerfrei wären, da bei diesen oft Ausnahmen, Randbedingungen und Eingabekonstellationen nicht vollständig berücksichtigt werden können. Dennoch gibt es Software, die über eine lange Zeit zuverlässig funktioniert. Selbst wenn beim Testen keine Fehler mehr zu Tage treten, heisst das noch nicht, dass die Software tatsächlich fehlerfrei sei.

### **2.1.1 Fehlerbegriff**

Anhand der Anforderungen und weiteren Informationen wird die *Testbasis* bestimmt, welche das erwartete Verhalten bezeichnet und als Grundlage für die Entscheidung dient, ob korrektes oder fehlerhaftes Verhalten vorliegt.

Ein *Fehler* ist somit eine festgestellte Abweichung zwischen dem festgelegten Sollverhalten und dem beobachteten Istverhalten. Solche Fehler entstehen nicht durch Alterung oder Verschleiss, sondern sind vom Zeitpunkt der Entwicklung an Teil der Software, auch wenn sie erst später entdeckt werden.

Wird die Fehlfunktion für den Anwender oder Tester sichtbar, spricht man von einer *Fehlerwirkung* (engl. *failure*). Zwischen Ursache, die ihren Ursprung im Fehlerzustand (engl. *fault*) der Software hat, und dem Auftreten der Fehlerwirkung muss unterschieden werden.

Ein Fehlerzustand kann durch andere Fehlerzustände in der Software kompensiert werden, was als *Fehlermaskierung* bezeichnet wird. Durch die Korrektur eines maskierenden Fehlers können bisher verborgene Fehlerzustände an den Tag treten. Ein Fehlerzustand kann erst verspätet oder andernorts in der Software zu einer Fehlerwirkung führen, etwa bei der Verfälschung gespeicherter Daten.

Fehlerzustände entstehen durch die *Fehlhandlung* einer Person (engl. *error*) und können verschiedene Gründe haben:

- Unvollkommenheit des Menschen
- hohe Komplexität von Aufgaben, Architektur, Design und Code
- hoher Zeitdruck
- Missverständnisse und Fehlinterpretationen der Anforderungen
- mangelnde Erfahrung oder Ausbildung der Beteiligten
- Ablenkung, Unkonzentriertheit und Müdigkeit

Eine Fehlhandlung einer Person führt zu einem Fehlerzustand im Programmcode, der zu einer Fehlerwirkung in der Software führt, die durch das Testen aufgezeigt werden soll.

Wird beim Testen eine Fehlerwirkung festgestellt, ohne dass ein Fehlerzustand im Testobjekt vorliegt, spricht man von einem *falsch positiven Ergebnis* (engl. *false positive result*). Wird bei vorhandenem Fehlerzustand keine entsprechende Fehlerwirkung entdeckt, liegt ein *falsch negatives Ergebnis* (engl. *false negative result*) vor. Wird der Fehlerzustand durch eine Fehlerwirkung erkannt, ist das Ergebnis *richtig positiv*; liegt kein Fehlerzustand vor und wird auch keine Fehlerwirkung erkannt, ist das Ergebnis *richtig negativ*.

Durch die Analyse der zugrundeliegenden Fehlhandlungen zu aufgedeckten Fehlerzuständen können Erkenntnisse gewonnen werden, womit der Entwicklungsprozess verbessert und die Wiederholung der Fehlhandlung vermieden werden kann.

Da zunächst nur die Fehlerwirkung und nicht der ihr zugrundeliegende Fehlerzustand bekannt ist, muss die fehlerhafte Stelle in der Software zuerst lokalisiert werden. Diesen Vorgang bezeichnet man als *Debugging*. Beim Testen werden also Fehlerwirkungen aufgedeckt, beim Debugging werden die zugrundeliegenden Fehlerzustände lokalisiert.

Durch die Behebung des Fehlerzustands wird die Qualität der Software verbessert – sofern bei dieser Korrektur keine neuen Fehlerzustände eingebaut werden. Ein erneuter Test nach der Fehlerkorrektur wird als *Fehlernachtest* bezeichnet. Da bei der Fehlerkorrektur aber auch neue Fehler eingebaut werden können, die unter anderen Eingabekonstellationen eine Fehlerwirkung erzeugen, müssen noch weitere Tests durchgeführt werden, und nicht nur derjenige, der die Fehlerwirkung ursprünglich provozierte.

### 2.1.2 Testbegriff

Beim Testen werden verschiedene Ziele verfolgt:

- qualitative Bewertung von Arbeitsergebnissen
- Nachweis der Anforderungserfüllung
- Bereitstellen von Informationen zur Einschätzung der Qualität des Testobjekts
- Verringerung der Risiken durch Aufdeckung und Beschreibung von Fehlerwirkungen
- Analyse der Artefakte zur Vermeidung und Erkennung von Fehlerwirkungen
- Erhalten von Informationen zum Testabdeckungsgrad

Diese Ziele unterscheiden sich je nach Entwicklungsmodell (klassisch, agil) und Teststufe. Geht es etwa beim Komponententest um das Aufdecken von Fehlerwirkungen, steht beim Abnahmetest die Erfüllung der Nutzererwartungen im Vordergrund, und ob das Produkt zur Nutzung freigegeben werden kann.

Die *Testbasis* legt das Sollverhalten des Testobjekts fest und umfasst neben Anforderungsdokumenten, User-Stories oder Spezifikationen auch voraussetzbares Fachwissen und den gesunden Menschenverstand. Das Ausführen des Testobjekts mit bestimmten Testdaten bezeichnet man als *Testfall*. Nach einem solchen *Testlauf* wird geprüft, ob eine Fehlerwirkung – eine Abweichung des beobachteten vom erwarteten Ergebnis – vorliegt. Aus einer Testbasis werden *Testbedingungen* abgeleitet, welche je von einem oder mehreren Testfällen überprüft werden.

Ein Testobjekt kann nicht als Ganzes sondern nur unterteilt in verschiedene Testelemente geprüft werden. (Testfälle prüfen einzelne Testelemente.) Testfälle werden in *Testsuiten* zusammengefasst und so in einem *Testzyklus* ausgeführt. Der *Testausführungsplan* legt die zeitliche Ausführung der Testsuiten fest. Ein *Testskript* automatisiert die Ausführung einer Testsuite und kümmert sich dabei um die Einhaltung der Vor- und Nachbedingungen bzw. Vorbereitungs- und Aufräumarbeiten. Bei manuellen Tests wird ein entsprechender schriftlicher Testablauf zur Verfügung gestellt.

Die Ergebnisse der Testläufe werden im *Testprotokoll* gesammelt und im *Testbericht* zusammenfassend dokumentiert. Zu Beginn werden Auswahl von Testobjekten und -verfahren sowie Teilziele und Testberichtserstattung im *Testkonzept*; die Koordination der Testaktivitäten im *Testzeitplan* festgelegt.

Je nach Art des Projekts kann der Aufwand für das Testen einen mehr oder weniger grossen Anteil am Gesamtaufwand ausmachen. Konnte man bei klassischen Projekten das Verhältnis von Testern zu Entwicklern zur Einschätzung des Testaufwands beziehen, fällt dies in agilen Projekten mit weniger starren Rollenverteilung schwer und kann höchstens anhand der Backlog-Aktivitäten grob abgeschätzt werden. Der Testaufwand muss ins Verhältnis zum Schadensausmass nicht gefundener Fehlerwirkungen und deren Eintretenswahrscheinlichkeit gestellt werden. ( $\text{Fehlerkosten} = \text{Auftretenswahrscheinlichkeit} \times \text{Schadensausmass}$ )

Ans Testen sollte in einem Projekt so früh wie möglich gedacht werden (*Shift-Left-Ansatz*: die Testaktivitäten werden auf der Zeitachse nach links verlegt). Bereits beim Überprüfen von Anforderungen und beim Refinement von User Stories können beteiligte Personen mit Testwissen früh mögliche Fehler erkennen und so vermeiden.

Das Risiko grundsätzlicher Konstruktionsfehler kann durch Beteiligung von Testern in der Designphase reduziert werden. In der Umsetzungsphase können Tester beim Vermeiden fehlerhafter Testfälle behilflich sein. Durch die Verifikation vor der Freigabe kann die Wahrscheinlichkeit erhöht werden, dass der Kunde ein Produkt erhält, das seinen Anforderungen entspricht.

### 2.1.3 Grundsätze des Testens

Beim Testen haben sich in den letzten Jahrzehnten die folgenden Grundsätze etabliert:

1. Das Testen zeigt die Anwesenheit von Fehlerzuständen, kann aber nicht deren Abwesenheit beweisen, selbst wenn keine Fehlerwirkungen gefunden werden.
2. Ein vollständiges Testen ist nicht bzw. nur bei den trivialsten Testobjekten möglich; Tests sind immer nur Stichproben.
3. Frühes Testen spart Zeit und Geld, da früh erkannte Fehler oft einfacher zu beheben sind als solche, die sich erst etwa im Produktivbetrieb auswirken.
4. Fehler sind nicht gleichmässig über das ganze System verteilt sondern treten gehäuft in wenigen Komponenten auf.
5. Testfälle müssen laufend erweitert werden, um neue Fehlerwirkungen erkennen zu können. Wiederholtes Ausführen bestehender Testfälle kann nur Regressionsfehler aufdecken.
6. Testen ist kontextabhängig und muss dem zu prüfenden System angepasst werden. Keine zwei Systeme können genau gleich geprüft werden.
7. Ein System kann unbrauchbar sein, selbst wenn keine Fehler darin gefunden werden. Es müssen auch Benutzbarkeit und Akzeptanz des Benutzers gegeben sein, am besten indem man diese früh in die Entwicklung miteinbezieht.

## 2.2 Softwarequalität: Qualitätsmanagement und Qualitätssicherung

Das *Qualitätsmanagement* umfasst organisatorische Tätigkeiten und Massnahmen der Lenkung und Leitung einer Organisation in Qualitätsfragen. Das Qualitätsmanagement ist Sache des Managements. Dieses legt Arbeitsprozesse fest, deren Einhaltung im Rahmen der *Qualitätssicherung* durch die jeweiligen Projektbeteiligten geprüft wird.

Testen wird oft mit Qualitätssicherung gleichgesetzt, ist aber nur eine Massnahme davon. Die Qualitätssicherung umfasst alle Tätigkeiten, womit die Qualität einer Komponente oder eines Systems bewertet wird.

Zur *Qualitätssteuerung* gehört auch die Analyse der Ursachen von Fehlern. In der Qualitätssicherung dienen Testergebnisse dazu, Verbesserungspotenzial im Prozess zu ermitteln; in der Qualitätssteuerung dienen sie zur Behebung von Fehlerzuständen.